

Docker Storage

Container-layer data • Named volumes • Bind mounts

1. Objective

This practical demonstrates Docker storage behavior in three cases: data written only to a container's writable layer, data stored in a Docker named volume, and data stored through a bind mount. The experiments show which data survives container deletion.

2. Understanding Docker Storage

A Docker container has a writable layer on top of the image's read-only layers. Files written only to this container layer belong to that container. If the container is deleted, those files are deleted with it. For data that must survive container replacement, Docker volumes or bind mounts should be used.

3. Demonstration 1 — Loss of Container-Layer Data

Create a temporary container and write data inside it:

```
docker run -it --name storage-test alpine sh
echo "This data is inside the container layer" > /data.txt
cat /data.txt
exit
```

Remove the container:

```
docker rm storage-test

# Create a new container with the same image
docker run --rm alpine cat /data.txt
```

The final command reports that /data.txt does not exist. The file was stored in the original container's writable layer, so deleting that container removed the data.

4. Demonstration 2 — Persistent Storage with a Named Volume

Create a Docker named volume:

```
docker volume create mydata
docker volume ls
```

Write data into the volume using a container:

```
docker run --rm -v mydata:/data alpine sh -c \
'echo "This data is stored in a named volume" > /data/data.txt'

docker run --rm -v mydata:/data alpine cat /data/data.txt
```

Remove the temporary container and create another container using the same volume:

```
docker run --rm -v mydata:/data alpine cat /data/data.txt

docker volume inspect mydata
```

The data remains available because the named volume exists independently of the container. The volume is not removed when a container using it is deleted.

5. Demonstration 3 — Persistent Storage with a Bind Mount

A bind mount maps a directory or file from the Docker host into the container. For this example, create a host directory named docker-storage-demo.

```
mkdir docker-storage-demo

docker run --rm \
```

```
-v "$(pwd)/docker-storage-demo:/data" \
alpine sh -c 'echo "This data is stored on the host" > /data/data.txt'

cat docker-storage-demo/data.txt
```

The file is physically available in the host directory. Start another container with the same bind mount:

```
docker run --rm \
-v "$(pwd)/docker-storage-demo:/data" \
alpine cat /data/data.txt
```

The second container can read the same file because the data is stored in the host filesystem rather than the container's writable layer.

6. Comparison

Storage type	Location	Survives container deletion?	Typical use
Container layer	Container writable layer	No	Temporary application state
Named volume	Docker-managed storage	Yes	Databases and persistent application data
Bind mount	Host filesystem	Yes	Development, configuration, host-shared files

7. Useful Volume Commands

```
docker volume ls
docker volume create NAME
docker volume inspect NAME
docker volume rm NAME

# Remove unused volumes
docker volume prune
```

Use `docker volume rm` only when the volume is no longer needed. Removing a named volume deletes the data stored in it.

8. Expected Results

Experiment 1 shows that container-layer data is lost when the container is deleted. Experiment 2 shows that a named volume preserves data independently of the container. Experiment 3 shows that a bind mount preserves data directly on the Docker host and makes it available to containers that mount the same host directory.

9. Conclusion

Docker's container writable layer is suitable for temporary data, but it should not be relied on for persistent storage. Named volumes are generally the preferred Docker-managed solution for persistent application data, while bind mounts are useful when the container needs direct access to files or directories on the host. Choosing the appropriate storage mechanism prevents accidental data loss when containers are recreated or deleted.